AAC-Assignment-14.4

Name: K. Sai Kumar Reddy

Batch No:42

2303A52223

Task 1: Responsive Homepage Layout for a Startup Website

Scenario

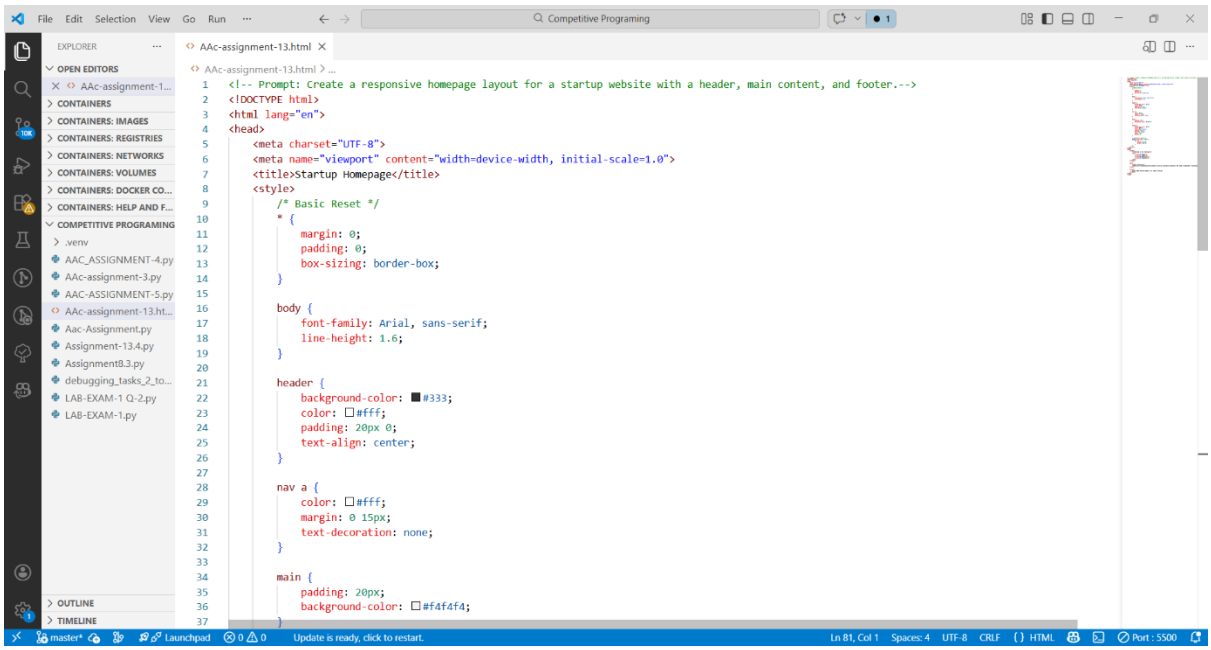
You are designing a **basic homepage for a startup company** that needs to look good on desktops, tablets, and mobile devices.

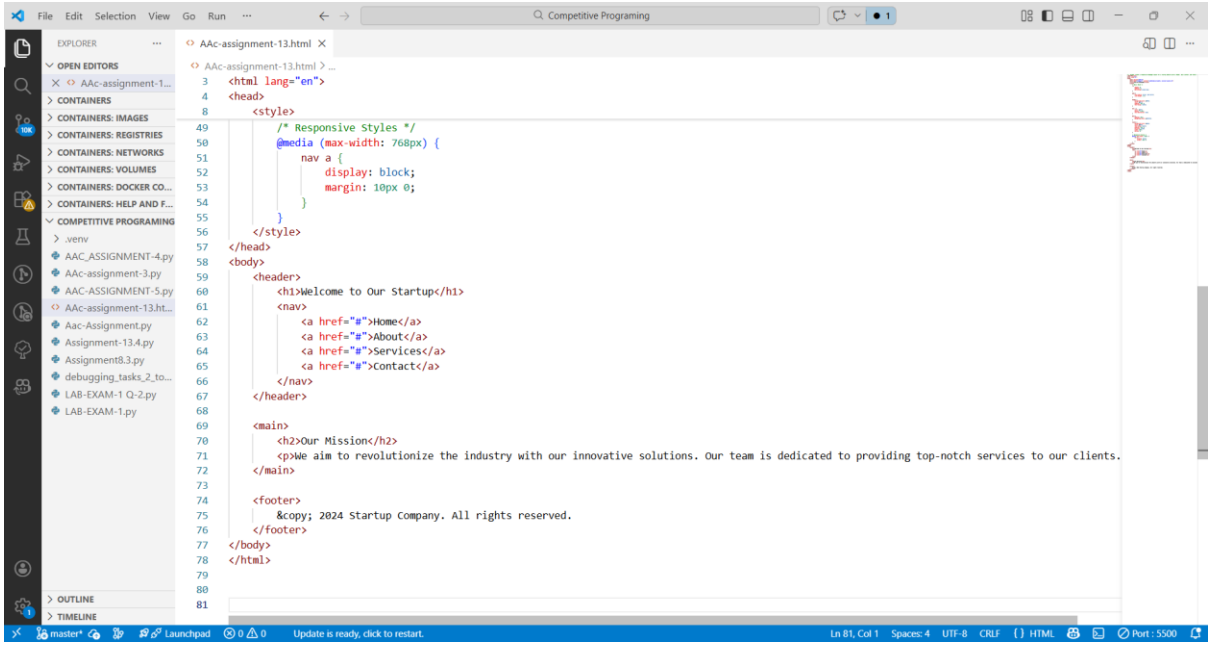
Task Description

- Create a web page structure using HTML that includes:
  - A header with navigation links
  - A main content section
  - A footer with basic information
- Use AI assistance to generate responsive CSS using media queries.
- Ensure the layout adjusts smoothly across different screen sizes.

Prompt: Create a responsive homepage layout for a startup website with a header, main content, and footer.

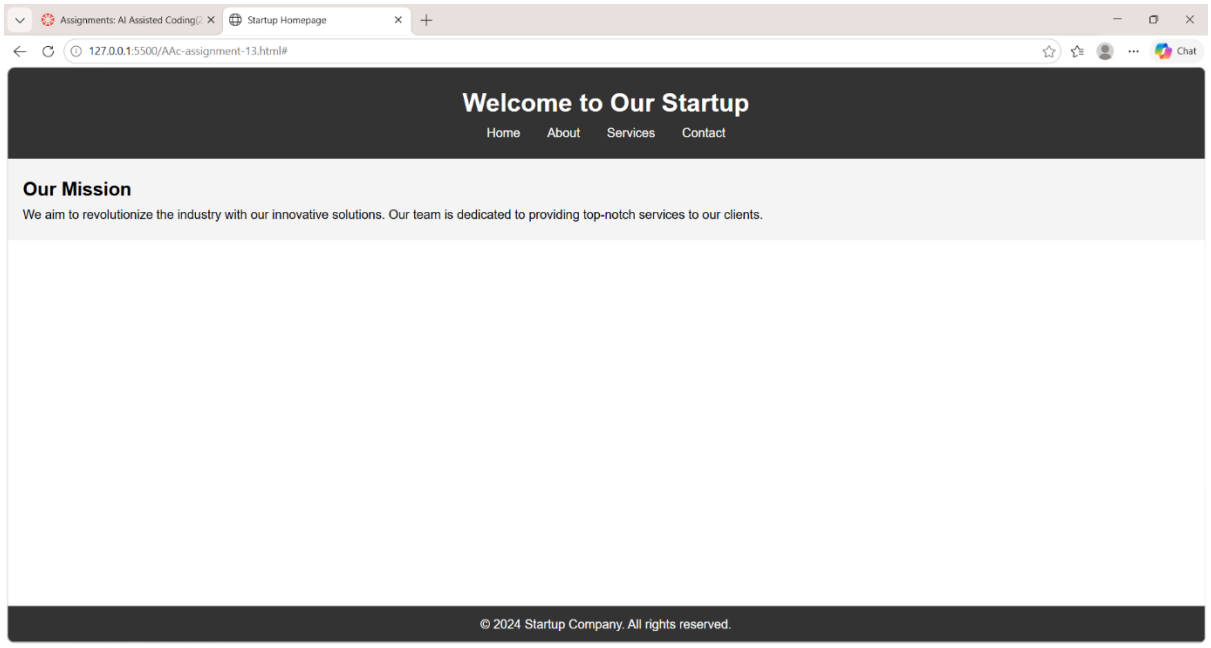
Implementation:





```html
<html lang="en">
<head>
    <style>
        /* Responsive Styles */
        @media (max-width: 768px) {
            nav a {
                display: block;
                margin: 10px 0;
            }
        }
    </style>
</head>
<body>
    <header>
        <h1>Welcome to Our Startup</h1>
        <nav>
            <a href="#">Home</a>
            <a href="#">About</a>
            <a href="#">Services</a>
            <a href="#">Contact</a>
        </nav>
    </header>

    <main>
        <h2>Our Mission</h2>
        <p>We aim to revolutionize the industry with our innovative solutions. Our team is dedicated to providing top-notch services to our clients.
    </main>

    <footer>
        &copy; 2024 Startup Company. All rights reserved.
    </footer>
</body>
</html>
```

Output:



Justification:

The layout is responsive and adapts to different screen sizes using media queries. The header, main content, and footer are clearly separated, and the design is clean and well-structured.

Task 2: Interactive Call-to-Action Button

Scenario

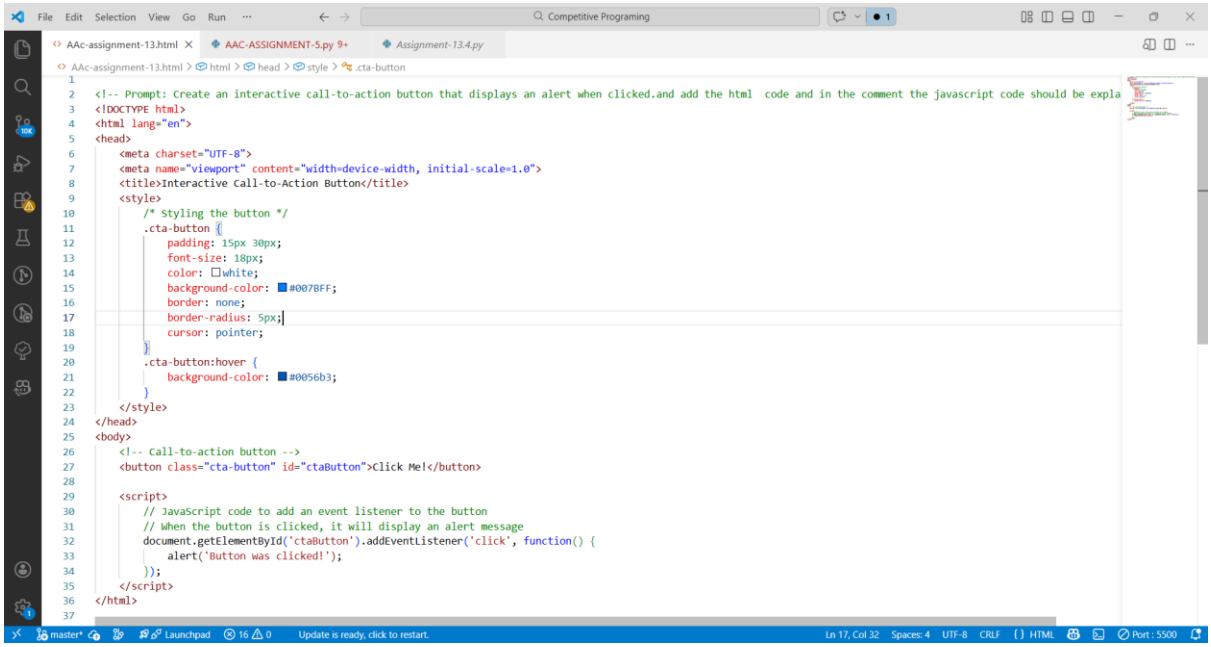
A marketing page requires a **call-to-action button** that responds when users interact with it.

Task Description

- Add a button to a web page using HTML.
- Use AI to generate JavaScript that responds to the button click by:
    - Displaying a message or alert
- Ask AI to add meaningful comments explaining the JavaScript logic.

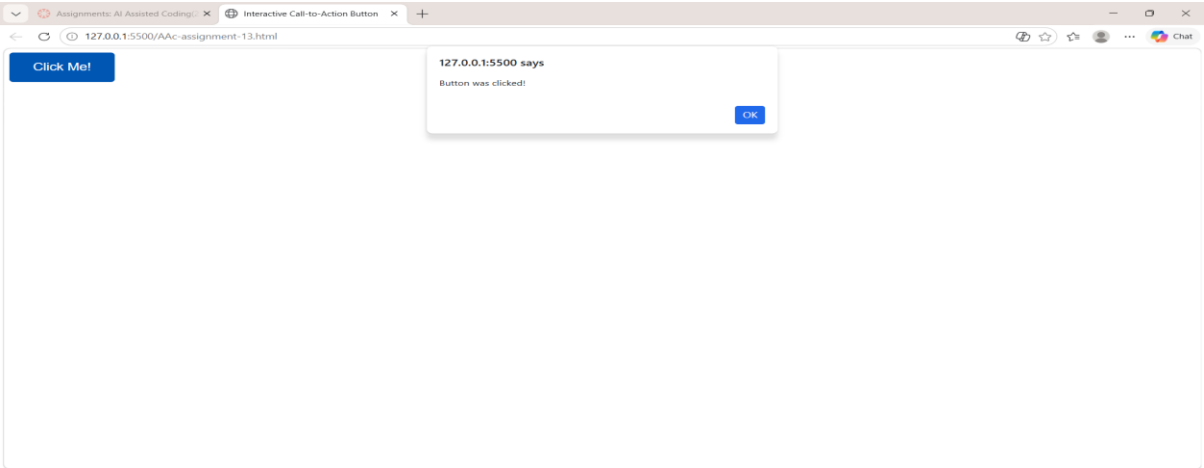
Prompt: Create an interactive call-to-action button that displays an alert when clicked.and add the html code and in the comment the Javascript code should be explained.

Implementation:



```html
<!-- Prompt: Create an interactive call-to-action button that displays an alert when clicked.and add the html  code and in the comment the javascript code should be expla
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Interactive Call-to-Action Button</title>
    <style>
        /* Styling the button */
        .cta-button {
            padding: 15px 30px;
            font-size: 18px;
            color: white;
            background-color: #007BFF;
            border: none;
            border-radius: 5px;
            cursor: pointer;
        }
        .cta-button:hover {
            background-color: #0056b3;
        }
    </style>
</head>
<body>
    <!-- Call-to-action button -->
    <button class="cta-button" id="ctaButton">Click Me!</button>

    <script>
        // JavaScript code to add an event listener to the button
        // When the button is clicked, it will display an alert message
        document.getElementById('ctaButton').addEventListener('click', function() {
            alert('Button was clicked!');
        });
    </script>
</html>
```

Output:



Justification:

The button is styled with CSS to make it visually appealing and interactive. When clicked, it triggers a JavaScript event listener that displays an alert message. This creates an engaging user experience by providing immediate feedback upon interaction.

Task 3: Contact Form with Client-Side Validation

Scenario

You are developing a **contact form for a company website** to collect user queries safely and correctly.

Task Description

- Design a form with fields for:
  - Name
  - Email
  - Message
- Use AI to generate JavaScript validation logic that:
  - Prevents empty submissions
  - Checks for a valid email format
- Display clear error messages for invalid input.

Prompt:

Generate a contact form using HTML and JavaScript with fields for Name, Email, and Message. Include client-side validation to prevent empty submissions, check for valid email format, and display clear error messages.

Code:

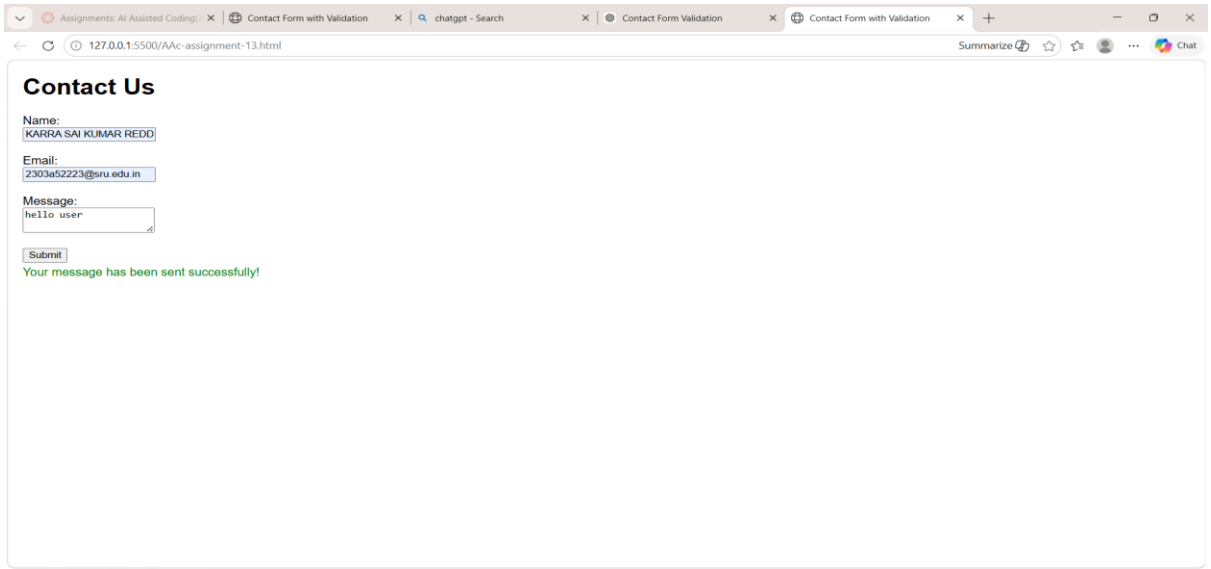
```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Contact Form with Validation</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
        }
        .error {
            color: red;
            margin-top: 5px;
        }
        .success {
            color: green;
            margin-top: 5px;
        }
    </style>
</head>
<body>
    <h1>Contact Us</h1>
    <form id="contactForm">
        <label for="name">Name:</label><br>
        <input type="text" id="name" name="name"><br><br>

        <label for="email">Email:</label><br>
        <input type="text" id="email" name="email"><br><br>

        <label for="message">Message:</label><br>
        <textarea id="message" name="message"></textarea><br><br>

        <button type="submit">Submit</button>
    </form>
```

```html
<html lang="en">
</head>
<body>
    <h1>Contact Us</h1>
    <form id="contactForm">
        <label for="name">Name:</label><br>
        <input type="text" id="name" name="name"><br><br>

        <label for="email">Email:</label><br>
        <input type="text" id="email" name="email"><br><br>

        <label for="message">Message:</label><br>
        <textarea id="message" name="message"></textarea><br><br>

        <button type="submit">Submit</button>
    </form>

    <div id="errorMessages" class="error"></div>
    <div id="successMessage" class="success"></div>

    <script>
        document.getElementById('contactForm').addEventListener('submit', function(event) {
            event.preventDefault(); // Prevent form submission

            // Clear previous messages
            document.getElementById('errorMessages').innerText = '';
            document.getElementById('successMessage').innerText = '';

            // Get form values
            const name = document.getElementById('name').value.trim();
            const email = document.getElementById('email').value.trim();
            const message = document.getElementById('message').value.trim();

            let errors = [];

            // Validate name
            if (name === '') {
```

```html
<html lang="en">
<body>
    <script>
        document.getElementById('contactForm').addEventListener('submit', function(event) {

            // Get form values
            const name = document.getElementById('name').value.trim();
            const email = document.getElementById('email').value.trim();
            const message = document.getElementById('message').value.trim();

            let errors = [];

            // Validate name
            if (name === '') {
                errors.push('Name is required.');
            }

            // Validate email
            const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
            if (email === '') {
                errors.push('Email is required.');
            } else if (!emailPattern.test(email)) {
                errors.push('Please enter a valid email address.');
            }

            // Validate message
            if (message === '') {
                errors.push('Message is required.');
            }

            // Display errors or success message
            if (errors.length > 0) {
                document.getElementById('errorMessages').innerText = errors.join(' ');
            } else {
                document.getElementById('successMessage').innerText = 'Your message has been sent successfully!';
                // Here you can also add code to actually send the form data to the server
                // For example, using fetch() or XMLHttpRequest
```

```html
<html lang="en">
<body>
    <script>
        document.getElementById('contactForm').addEventListener('submit', function(event) {
            // Validate email
            const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
            if (email === '') {
                errors.push('Email is required.');
            } else if (!emailPattern.test(email)) {
                errors.push('Please enter a valid email address.');
            }

            // Validate message
            if (message === '') {
                errors.push('Message is required.');
            }

            // Display errors or success message
            if (errors.length > 0) {
                document.getElementById('errorMessages').innerText = errors.join(' ');
            } else {
                document.getElementById('successMessage').innerText = 'Your message has been sent successfully!';
                // Here you can also add code to actually send the form data to the server
                // For example, using fetch() or XMLHttpRequest
            }
        });
    </script>
</body>
</html>
```

Output:



Justification:

Client-side validation is important to ensure that user input is correct before it is sent to the server. By validating fields such as Name, Email, and Message, the system prevents empty or incorrect data from being submitted. The email format check ensures that the user enters a valid email address, which helps maintain data quality. Displaying clear error messages improves user experience by guiding users to correct their input immediately. This approach also reduces unnecessary server requests and improves the overall efficiency and reliability of the web application.

Task 4: Dynamic List Management for a Product Page

Scenario

A product listing page needs to **add or remove items dynamically** without refreshing the page.

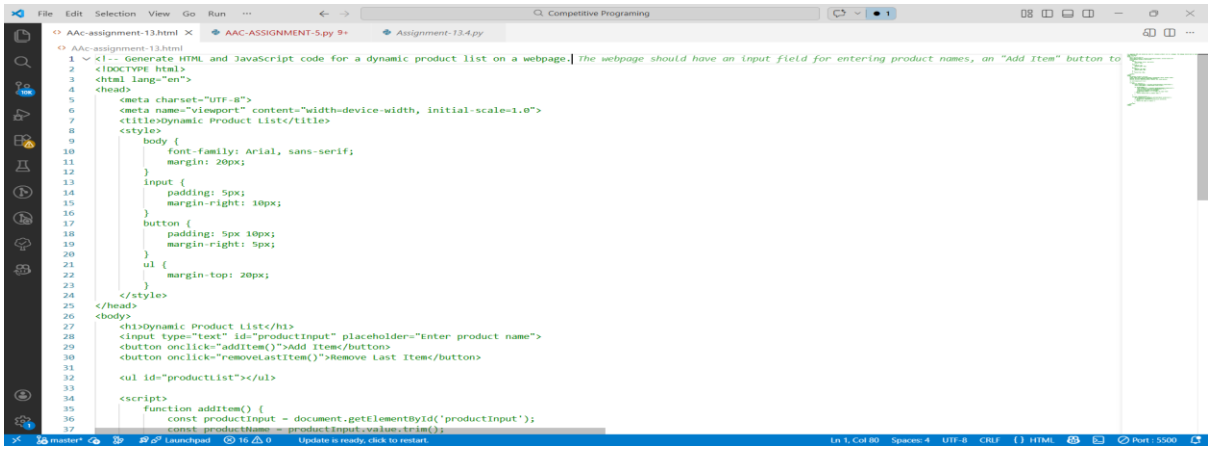
Task Description

- Create a simple HTML list to display items.
- Use AI to generate JavaScript that:
    - Adds a new item when an “Add” button is clicked
    - Removes the last item when a “Remove” button is clicked
- Ensure the DOM updates dynamically.

Prompt:

Generate a simple HTML product list with JavaScript that allows users to dynamically add items to the list and remove the last item using "Add" and "Remove" buttons without refreshing the page.

Code:



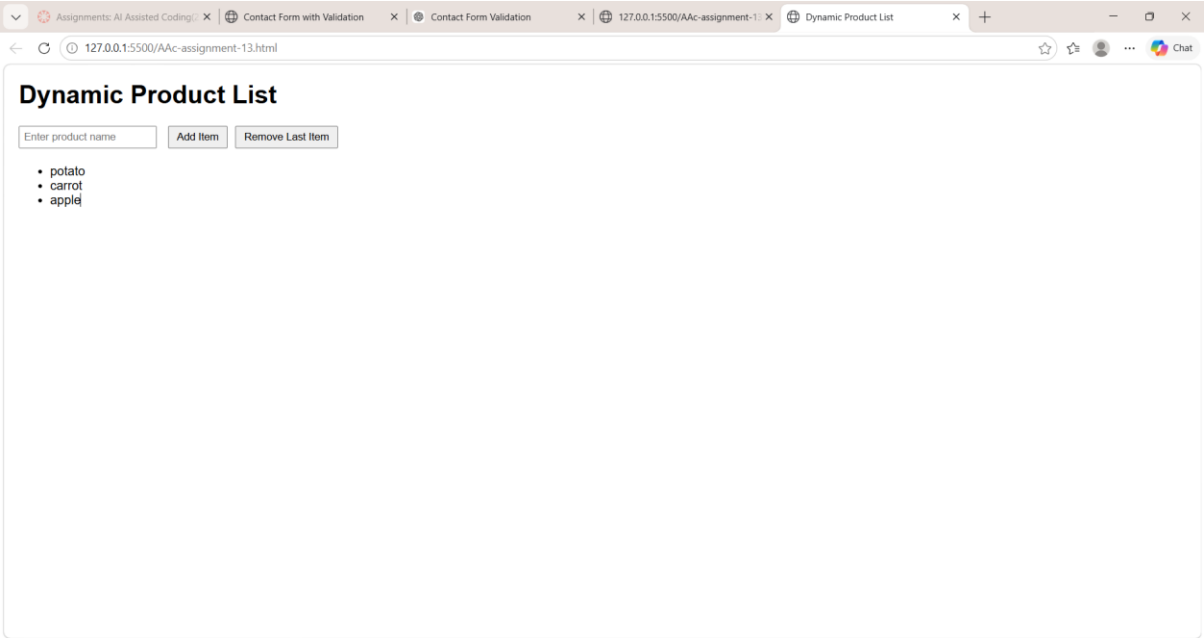
AAc-assignment-13.html

```html
            margin-top: 20px;
        }
    </style>
</head>
<body>
    <h1>Dynamic Product List</h1>
    <input type="text" id="productInput" placeholder="Enter product name">
    <button onclick="addItem()">Add Item</button>
    <button onclick="removeLastItem()">Remove Last Item</button>

    <ul id="productList"></ul>

    <script>
        function addItem() {
            const productInput = document.getElementById('productInput');
            const productName = productInput.value.trim();

            if (productName) {
                const productList = document.getElementById('productList');
                const listItem = document.createElement('li');
                listItem.textContent = productName;
                productList.appendChild(listItem);
                productInput.value = ''; // Clear the input field
            } else {
                alert('Please enter a product name.');
            }
        }

        function removeLastItem() {
            const productList = document.getElementById('productList');
            if (productList.lastChild) {
                productList.removeChild(productList.lastChild);
            } else {
                alert('No items to remove.');
            }
        }
    </script>
```

AAc-assignment-13.html

```html
    <button onclick="removeLastItem()">Remove Last Item</button>

    <ul id="productList"></ul>

    <script>
        function addItem() {
            const productInput = document.getElementById('productInput');
            const productName = productInput.value.trim();

            if (productName) {
                const productList = document.getElementById('productList');
                const listItem = document.createElement('li');
                listItem.textContent = productName;
                productList.appendChild(listItem);
                productInput.value = ''; // Clear the input field
            } else {
                alert('Please enter a product name.');
            }
        }

        function removeLastItem() {
            const productList = document.getElementById('productList');
            if (productList.lastChild) {
                productList.removeChild(productList.lastChild);
            } else {
                alert('No items to remove.');
            }
        }
    </script>
</body>
</html>
```

Output:



Justification:

Dynamic list management allows web pages to update content without reloading the page. By using **JavaScript DOM manipulation**, items can be added or removed from the product list instantly when the user clicks the **Add** or **Remove** buttons. This improves interactivity and provides a smoother user experience. It also demonstrates how modern web applications handle real-time updates efficiently, which is commonly used in e-commerce platforms and interactive websites.

Task 5: Modal Popup for User Notifications

Scenario

You are building a **user notification popup** for announcements or alerts on a website.

Task Description

- Use AI to help generate:
    - HTML structure for a modal popup
    - CSS styling with a semi-transparent background overlay
    - JavaScript to open and close the modal

Include multiple ways to close the modal (button or overlay click).

Prompt:

Generate simple HTML, CSS, and JavaScript code for a modal popup notification on a webpage. Include a semi-transparent background overlay and a button to open the modal. The modal should close when the user clicks the close button or the overlay.

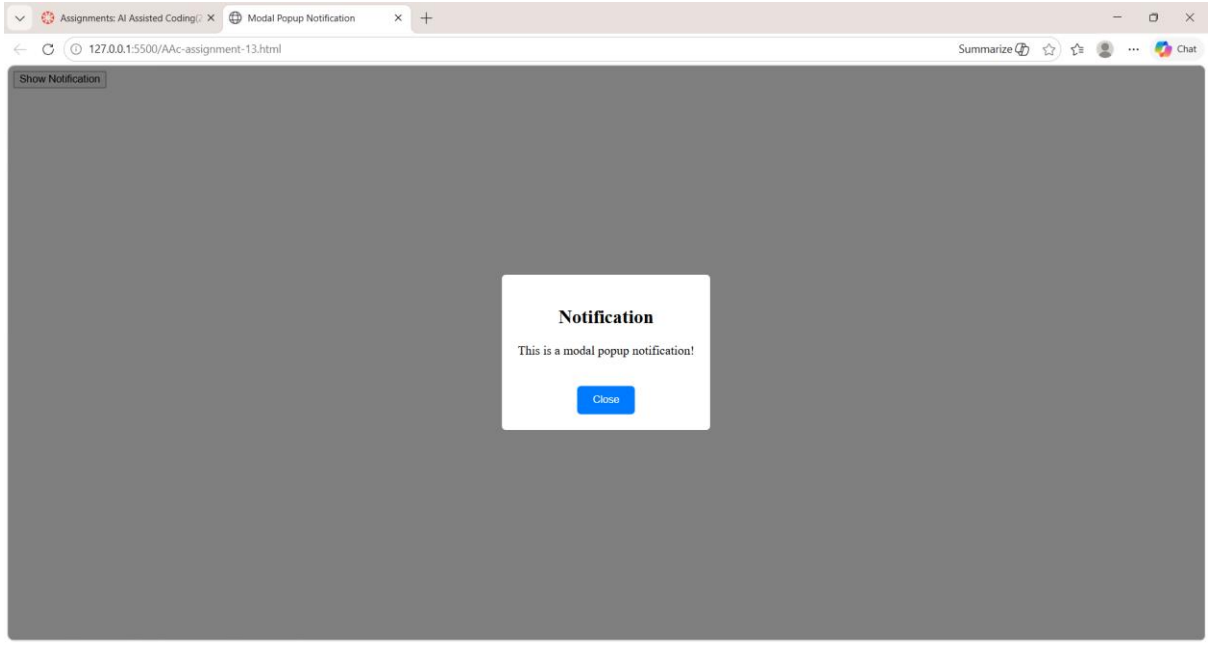
Code:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Modal Popup Notification</title>
    <style>
        /* Style for the modal background overlay */
        .modal-overlay {
            position: fixed;
            top: 0;
            left: 0;
            width: 100%;
            height: 100%;
            background-color: rgba(0, 0, 0, 0.5); /* Semi-transparent background */
            display: none; /* Hidden by default */
            justify-content: center;
            align-items: center;
        }

        /* Style for the modal content */
        .modal-content {
            background-color: white;
            padding: 20px;
            border-radius: 5px;
            text-align: center;
        }

        /* Style for the close button */
        .close-button {
            margin-top: 20px;
            padding: 10px 20px;
            background-color: #007BFF;
            color: white;
            border: none;
            border-radius: 5px;
            cursor: pointer;
```

```html
<html lang="en">
<head>
    <style>
        /*
        </style>
</head>
<body>
    <!-- Button to open the modal -->
    <button id="openModalButton">Show Notification</button>

    <!-- Modal structure -->
    <div id="modalOverlay" class="modal-overlay">
        <div class="modal-content">
            <h2>Notification</h2>
            <p>This is a modal popup notification!</p>
            <button id="closeModalButton" class="close-button">Close</button>
        </div>
    </div>

    <script>
        // Get references to the modal elements
        const openModalButton = document.getElementById('openModalButton');
        const modalOverlay = document.getElementById('modalOverlay');
        const closeModalButton = document.getElementById('closeModalButton');

        // Function to open the modal
        function openModal() {
            modalOverlay.style.display = 'flex'; // Show the modal
        }

        // Function to close the modal
        function closeModal() {
            modalOverlay.style.display = 'none'; // Hide the modal
        }

        // Event listener for opening the modal
        openModalButton.addEventListener('click', openModal);
```

```html
<html lang="en">
<body>

    <script>
        // Get references to the modal elements
        const openModalButton = document.getElementById('openModalButton');
        const modalOverlay = document.getElementById('modalOverlay');
        const closeModalButton = document.getElementById('closeModalButton');

        // Function to open the modal
        function openModal() {
            modalOverlay.style.display = 'flex'; // Show the modal
        }

        // Function to close the modal
        function closeModal() {
            modalOverlay.style.display = 'none'; // Hide the modal
        }

        // Event listener for opening the modal
        openModalButton.addEventListener('click', openModal);

        // Event listener for closing the modal when clicking the close button
        closeModalButton.addEventListener('click', closeModal);

        // Event listener for closing the modal when clicking outside the modal content
        window.addEventListener('click', function(event) {
            if (event.target === modalOverlay) {
                closeModal();
            }
        });
    </script>
</body>
</html>
```

Output:



Justification:

A modal popup is commonly used on websites to display important notifications, announcements, or alerts without navigating to another page. The modal appears on top of the existing content with a semi-transparent overlay, which focuses the user's attention on the message. Using JavaScript to open and close the modal dynamically improves user interaction and avoids page reloads. Providing multiple ways to close the modal, such as clicking a close button or the overlay, enhances usability and accessibility. This approach is widely used in modern web applications to deliver information effectively while maintaining a smooth user experience